

Code Metrics Lab

A browser-native text & code analyser — no server, no database.

Paste any text or code and read it back as live metrics, conversions and validations — everything computed in the browser as you type.

Pega cualquier texto o código y léelo como métricas, conversiones y validaciones en vivo — todo calculado en el navegador mientras escribes.

Team · Deodanes · Mendoza · Mata · Rodríguez **Environment** · Vue 3 (CDN)

Institution · INDEL — BTV Software Development **Date** · 2026-08-28

Vue 3

HTML / CSS

localStorage

no build step

100% English

The environment used

Code Metrics Lab runs on **Vue 3 in CDN mode**: a single `index.html` loads Vue from a script tag, with a local copy in `/assets` as an offline fallback. The tool is one interactive page with no accounts and no server data, so full reactivity with zero build tooling is exactly the right weight — the project opens and runs anywhere just by opening the file.

Code Metrics Lab funciona con Vue 3 en modo CDN: un solo index.html carga Vue desde una etiqueta script, con una copia local en /assets como respaldo sin conexión. La herramienta es una página interactiva sin cuentas ni datos de servidor, así que la reactividad completa sin compilación es justo el peso correcto.

Why this fits

Live results as the user types, three readable files, and nothing to install or compile.
Portable and easy to review end to end.

Alternative not used — Vite + npm

Gives bundling and hot reload, but adds `node_modules`, a build step and config this single page does not need. CDN is simpler and more portable here.

Tools

Tool	Version	What it was used for
Visual Studio Code	1.x	Main editor for HTML, CSS and JavaScript.
Live Server (VS Code ext.)	—	Local dev server at 127.0.0.1:5500 to run and reload the app.
Vue 3 (CDN global build)	3.5.42	Reactivity: live results via computed properties.
Git & GitHub	—	Version control and public repository.
Microsoft Edge	—	Browser used to run and test the app.

Resources

Resource	Detail	Contribution
Hardware	Personal laptop / lab PC (Windows 11)	Development and testing.
Time invested	~3 blocks of 45 min across the activity	Build, debug and document.
Documentation	vuejs.org · MDN (String / JSON) · Live Server docs	Reactivity, string handling, JSON parsing.
Libraries	Vue 3 only (via CDN) — no other libraries	Core reactivity; minimal dependencies.
AI / assistance	AI assistants (Claude, ChatGPT, Gemini) to learn techniques, draft docs and generate test inputs	Guidance and learning; all code reviewed, run and tested by the team.

Se declara el uso de asistentes de IA para aprender técnicas, redactar documentación y generar entradas de prueba. Todo el código fue revisado, ejecutado y probado por el equipo.

Functionality

Seven tools share one editor. The user types once; each tool reads the same text and returns its own live result.

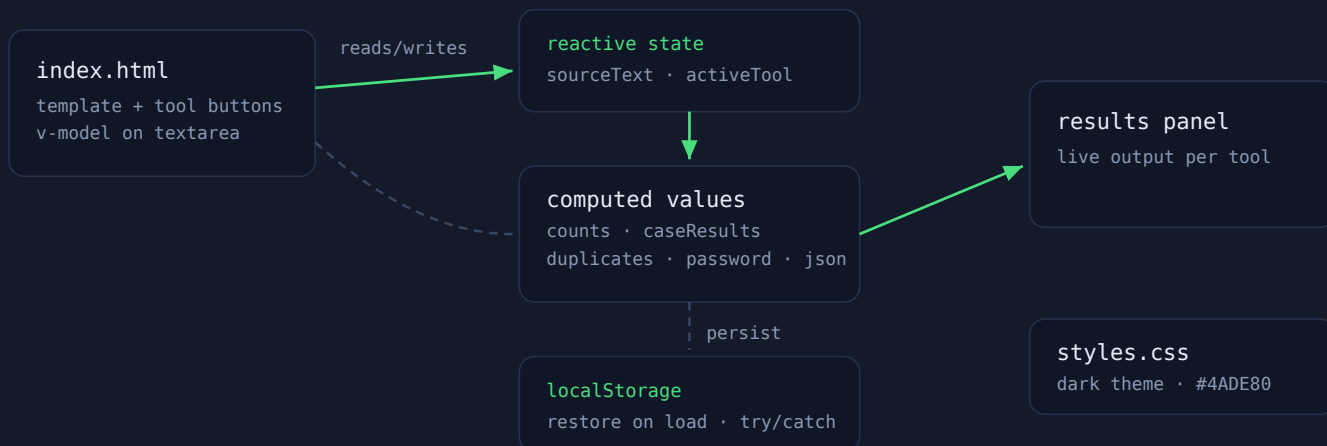
Feature	What the user does and sees	Evidence
Live counter	Words, characters, lines, paragraphs and reading time update instantly.	EV-02
Case converter	Text in UPPERCASE, lowercase, Title Case, snake_case and camelCase at once.	EV-03
Duplicate lines	Which lines repeat and how many times.	EV-04
Password strength	Five criteria pass/fail and a Weak / Medium / Strong level.	EV-05
JSON formatter	Pretty-prints valid JSON or shows a clear error for invalid JSON.	EV-06
Persistence	Reloads the page and the text and active tool are restored.	EV-07
Clear	Empties the text and the saved data with one button.	EV-08

Deliberately out of scope: no files saved to disk, no database, no server, no login. It analyses only the text in the editor. Everything stays in the browser — private by design.

Fuera de alcance a propósito: no guarda archivos, no usa base de datos ni servidor, no requiere login.

Architecture & data model

Three files, one reactive state, and computed values that do all the work. The interface only reads the state — a clean line between what shows and what knows.



La interfaz lee el estado; las computed calculan; el resultado se muestra y el estado se guarda en localStorage.

Folders

/ index.html · /css styles · /js app.js ·
/assets local Vue fallback.

Persistence — no database

Data lives in memory. `sourceText` and `activeTool` are mirrored to `localStorage` and restored on load; every access is wrapped in `try/catch`.

Code highlights

```
// A paragraph is a block separated by one or more blank lines.
paragraphCount() {
  return this.sourceText
    .split(/\n\s*\n/)
    .map((block) => block.trim())
    .filter((block) => block.length > 0).length;
}
```

Interesting because it encodes a real definition — a paragraph is blank-line-separated, not just any newline — so multi-line text counts the way a human reads it.

```
// Group equal trimmed lines; return those seen more than once.
duplicateLines() {
  const counts = new Map(), order = [];
  this.sourceText.split(/\r?\n/).forEach((raw) => {
    const line = raw.trim();
    if (line === '') return; // ignore blank lines
    if (!counts.has(line)) order.push(line);
    counts.set(line, (counts.get(line) || 0) + 1);
  });
  return order.filter((l) => counts.get(l) > 1)
    .map((l) => ({ line: l, count: counts.get(l) }));
}
```

Interesting because it preserves first-appearance order and ignores blank lines and surrounding spaces — small rules that make the output trustworthy.

```
// Parsing is wrapped so no input can ever crash the app.
jsonResult() {
  const text = this.sourceText.trim();
  if (text === '') return { state: 'empty' };
  try {
    const parsed = JSON.parse(text);
    return { state: 'valid', output: JSON.stringify(parsed, null, 2) };
  } catch (error) {
    return { state: 'invalid', error: error.message };
  }
}
```

Interesting because it is the core robustness idea: try/catch turns any bad input into a controlled message instead of a crash.

Testing

Nine cases, written while testing. One real failure (T-03) was found, documented and fixed — the re-test T-04 passes.

9

test cases

8

PASS

1

FAIL found & fixed

7/7

MUST features

ID	What was tested	Input	Expected	Actual	Status
T-01	Live counters, multi-paragraph	3 paragraphs, 79 words	Correct counts	Words 79 · Lines 6 · Paragraphs 3	PASS
T-02	Counters after Clear	Empty text	All at 0	All at 0	PASS
T-03	snake_case / camelCase multi-line	5 lines "Hello World"	Convert each line, keep 5 lines	Joined all lines into one string	FAIL
T-04	Re-test after fix	5 lines "Hello World"	5 separated lines	5 lines kept separate	PASS
T-05	Duplicate line finder	2 lines "Hello World"	Report count	"Hello World" — 2 times	PASS
T-06	Password strength	upper+lower+digit+symbol	Strong, 5/5	Strong, 5/5	PASS
T-07	JSON valid	Correct object	Valid, indented	Valid JSON, 2-space indent	PASS
T-08	JSON invalid	Trailing comma	Clear error + position	Invalid JSON: ... position 28	PASS
T-09	Persistence	Type + reload	Text & tool restored	Restored correctly	PASS

Challenges & conclusions

Problem	Searched in English	How it was solved
snake_case / camelCase joined all lines into one string	"javascript convert each line separately"	Processed the text line by line so each line yields its own result.
Browser auto-translated the UI, looking like Spanish in the code	"disable Edge automatic translation for a site"	Confirmed source is fully English; turned off page translation for evidence.
JSON.parse could crash the tool	"javascript json.parse try catch error message"	Wrapped parsing in try/catch with a controlled English error and position.

A small reactive front end can deliver real utility with almost no infrastructure, and separating state from computed logic made each feature easy to add and test. Next time we would plan the microcopy and test cases earlier and pick the accent colour before styling. With one more week: file drag-and-drop, more converters and export-to-file — still with no database.

Un front-end pequeño y reactivo puede dar utilidad real casi sin infraestructura. Con una semana más: arrastrar archivos, más convertidores y exportar a archivo, siempre sin base de datos.